TASK 5 – Organise the Program with Header Files and Conditional Compilation 3 Marks Bloom's Level: K3 – Apply Organise the constants and function prototypes into a header file and add a debugging mode.

Discuss:

• The structure of a C program: header files, global declarations, main() and other functions

• Include guards (#ifndef ... #define ... #endif) and why they are needed

• Conditional compilation with #ifdef DEBUG to print intermediate values only when compiled with gcc -DDEBUG

• The difference between #include and #include "file" Alternative Concept and Complexity Analysis If debugging output were controlled by a run-time variable (if (debug) printf(...)) instead of #ifdef DEBUG, what would the solution be? Analyse the time cost of the extra test in every call and the space taken by debug code that remains in the executable in each approach. Evidence Expected A header file with include guards, a working DEBUG mode and an explanation of the program structure.

1. Structure of a C Program

A C program can be organised into separate files to improve readability, reuse and maintenance.

- **Header files:** Contain constants, macros, type definitions and function prototypes.
- **Global declarations:** Define items that must be accessible to multiple functions.
- **main() function:** Acts as the entry point of the program and controls execution.
- **Other functions:** Perform specific operations, such as calculating the electricity bill.

Separating declarations from implementations makes a program easier to maintain and test.

2. Include Guards

Include guards prevent the same header file from being processed multiple times in one compilation unit.

Example:

#ifndef BILLING_H

```
#define BILLING_H

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units);

#endif
```

Here, #ifndef BILLING_H checks whether BILLING_H has not been defined. If it has not, the contents are processed and the macro is defined. Later inclusions of the same header are skipped.

This prevents duplicate declarations and related compilation problems.

3. Conditional Compilation Using DEBUG

Conditional compilation allows debugging statements to be included only when required.

Example:

```
#ifdef DEBUG

printf("Units consumed: %d\n", units);

#endif
```

Compile normally:

```
gcc main.c billing.c -o billing
```

Compile with debugging enabled:

```
gcc -DDEBUG main.c billing.c -o billing
```

When DEBUG is defined, the debugging statements are included in the compiled program. Without DEBUG, those statements are removed during preprocessing.

This makes it possible to inspect intermediate values during development without displaying them in the normal program output.

4. Difference Between Angle Brackets and Double Quotes

Syntax	Purpose

#include <stdio.h>	Normally searches the compiler's system header directories.
#include "billing.h"	Normally searches the current source file's directory first, followed by other configured include directories.

Angle brackets are commonly used for standard library headers. Double quotes are commonly used for project-specific header files.

5. Alternative Concept: Run-Time Debugging

Instead of using #ifdef DEBUG, debugging could be controlled by a run-time variable:

```
if (debug)
  printf("Units consumed: %d\n", units);
```

In this approach, the program checks the value of debug whenever the statement is reached. The debugging code remains in the executable even when debugging is disabled at run time.

With #ifdef DEBUG, the compiler includes the debugging code only when the macro is defined. If it is not defined, the debugging code is excluded during preprocessing.

6. Time and Space Complexity Analysis

Factor	**#ifdef DEBUG**	**Run-time if (debug)**
Extra run-time test	None when DEBUG is disabled	A condition check whenever execution reaches the statement
Debug code in executable	Excluded when DEBUG is not defined	Normally remains in the executable
Execution time	No debug-statement overhead when excluded	Small condition-check overhead
Code size	Usually smaller without debug statements	May be larger because debug statements remain

The time complexity of the billing calculation itself is unchanged. Conditional compilation mainly affects the compiled code and debugging overhead, rather than the billing algorithm's complexity.

Code :

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
  long paise = FIXED_CHARGE_PAISE;

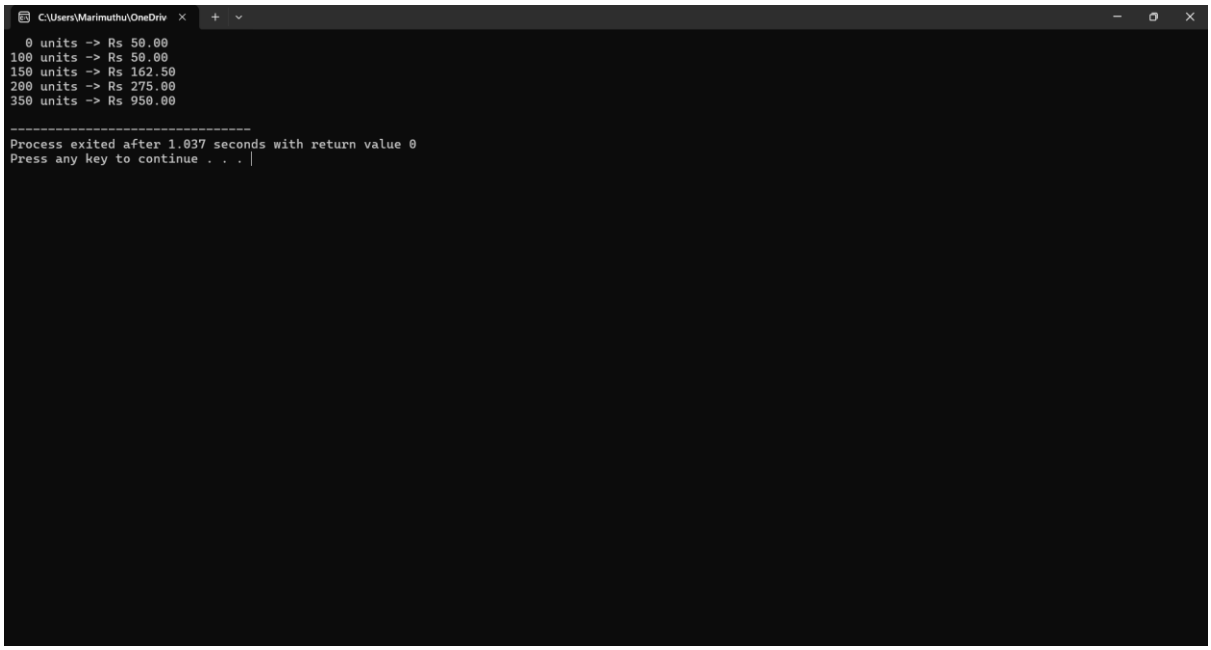
  if (units > SLAB2_UNITS) {
    paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
  }
  if (units > SLAB1_UNITS) {
    int slab2;
    if (units < SLAB2_UNITS)
      slab2 = units - SLAB1_UNITS;
    else
      slab2 = SLAB2_UNITS - SLAB1_UNITS;
    paise += (long)slab2 * RATE2_PAISE;
  }

  if (units > 0 && units <= SLAB1_UNITS)
```

```
    paise += (long)units * RATE1_PAISE;
#ifdef DEBUG
  printf("[DEBUG] Units = %d, Bill = %ld paise\n",
      units, paise);
#endif
  return paise;
}
int main(void)
{
  int tests[] = {0, 100, 150, 200, 350};
  int n = sizeof(tests) / sizeof(tests[0]);
  int i;

  for (i = 0; i < n; i++) {
    long p = compute_bill(tests[i]);
    printf("%3d units -> Rs %ld.%02ld\n",
        tests[i], p / 100, p % 100);
  }
  return 0;
}
```

Output:



7. Conclusion

Header files organise declarations and constants, while include guards prevent repeated inclusion. Conditional compilation using #ifdef DEBUG allows debugging statements to be included only when needed. This approach improves program organisation, maintainability and control over the final executable.